

Riepilogo del lavoro: dai dati SUMO al replanning

Documento riassuntivo di tutte le estensioni aggiunte al progetto di navigazione urbana in PDDL+ per Dublino. Copre i quattro punti della roadmap (estrazione, ottimizzazione, iniezione, confronto) e la funzione di replanning nella webapp. Ogni sezione spiega **cosa fa**, **come funziona** e **perché è stato fatto così**. In coda, la sezione "In sintesi".

Punto 1 — Estrazione dei dati da SUMO

File: `scripts/extract_sumo_data.py` → produce `sumo_extracted/sumo_data_<zona>.json`

Cosa fa

Legge le reti stradali di SUMO (`net_files/*.net.xml`) ed estrae, per ogni zona, due famiglie di dati che il modello PDDL+ non aveva:

- 1 **Semafori e loro tempi** — da ogni elemento `<tlLogic>` (il programma semaforico di un incrocio) legge le fasi con durata e stato (verde/giallo/rosso).
- 2 **Turnrate** — per ogni svolta possibile a un incrocio, l'angolo di cui il veicolo deve ruotare.

Come funziona

Semafori. I tempi presenti nel `net.xml` non sono quelli reali di Dublino: sono i valori *di default* generati da `netconvert` (ciclo fisso di 90 s, verde diviso equamente, giallo calcolato dalla velocità della strada). A Dublino gli incroci sono controllati dal sistema adattivo **SCATS**, con cicli tipici fino a ~120 s. Per ottenere un ritardo realistico si procede così:

- si prende da SUMO la **struttura** dell'incrocio (quali movimenti sono verdi insieme e la proporzione verde/rosso, che dipende dalla geometria reale);
- si **riscalda il ciclo** dai 90 s di SUMO ai 120 s realistici di Dublino;
- si calcola l'attesa media di un veicolo con la **formula di Webster**: $\text{ritardo} = \text{rosso}^2 / (2 \cdot \text{ciclo})$.

Esempio: un incrocio a 2 fasi ha il verde per ~47 % del ciclo, quindi rosso reale ≈ 64 s su 120 s, da cui ritardo $\approx 64^2 / (2 \cdot 120) \approx \mathbf{17\ s}$. È questo il valore che sostituisce i 30 s fissi usati in precedenza.

Turnrate. Per turn rate si intende la velocità angolare con cui un veicolo cambia direzione, in gradi al secondo. Si usa 20 °/s (yaw rate reale di un'auto in svolta urbana stretta). Il tempo di svolta è $|\text{angolo di svolta}| / \text{turn_rate}$, dove l'angolo è la differenza fra la direzione della strada in ingresso e quella in uscita all'incrocio. Esempi: svolta a 90° → 4,5 s; inversione a U → 9 s; proseguire dritto → ~0 s.

Integrazione nel modello PDDL+

Questi dati sono poi entrati nel dominio:

- `build_problems.py` e la **webapp** scrivono i ritardi semaforici realistici (al posto dei 30 s fissi) e i fatti `turn-time` per ogni tripla di nodi consecutivi.

- Il **dominio** (`pddl_files/domain.pddl`) è stato esteso: l'azione `start-move` ora conosce il nodo di provenienza (`?prev`) e paga il tempo di svolta all'incrocio; un fatto `prev` tiene traccia dell'ultimo arco percorso.

Perché conta (e un effetto collaterale)

Il modello diventa molto più realistico: i tempi non sono più inventati. Il prezzo di questa scelta è che sia `start-move` sia `signal-delay` diventano funzioni a **tre argomenti** (`prev` , `from` , `to`): il numero di istanze che ENHSP deve generare cresce quindi molto rapidamente col numero di nodi. È la ragione dei limiti di dimensione discussi più avanti.

Punto 2 — Ottimizzazione semaforica

File: `cartella signal_optimization/` → produce `sumo_extracted/signal_plan_<zona>.json`

Cosa fa

Cerca durate di verde migliori per i semafori, con l'obiettivo di ridurre il tempo di percorrenza medio su un campione di percorsi. Usa il modello PDDL+ già costruito come funzione di valutazione: per ogni configurazione candidata misura il `total-time` medio con ENHSP e tiene la migliore.

Come funziona (in breve)

Pipeline in più fasi: generazione di candidati vincolati → screening analitico con Webster → validazione con ENHSP → confronto baseline vs ottimizzato sul campione O-D condiviso. L'output è, per ogni zona, un file `signal_plan_<zona>.json` nel formato `{tlLogic_id: {fase: durata}}` .

Risultato stimato

Zona	Semafori ottimizzati	<code>total-time</code> medio: baseline → ottimizzato
piccola	1	16,45 s → 10,33 s (−37 %)
media	3	72,04 s → 47,07 s (−35 %)
grande	16	97,80 s → 67,94 s (−31 %)

Nota: i semafori *ottimizzati* sono pochi rispetto al totale, perché la ricerca agisce solo sulle giunzioni realmente attraversate dal campione di traffico.

Punto 3 — Iniezione del piano in SUMO

File: `scripts/inject_signal_plan.py` → produce `cfg_files/tls_<zona>.add.xml`

Cosa fa

Traduce il piano ottimizzato del punto 2 in un formato che SUMO sa caricare, chiudendo il ciclo PDDL+ → SUMO. Genera un *additional-file* con i `<tlLogic>` che hanno le durate di fase ottimizzate.

Come funziona

Legge il `net.xml` e il `signal_plan_<zona>.json`, poi riscrive ogni semaforo conservando dall'originale gli stati delle fasi (`GGrrrr` ...), l'offset e il tipo, e sovrascrivendo **solo** le durate presenti nel piano.

Il punto tecnico chiave, verificato sulla documentazione SUMO: "*when loaded, the last program will be used*". Basta quindi che il programma abbia un `programID` **nuovo** (`optimized`, diverso dall'originale `0`) e SUMO lo rende attivo da solo, senza bisogno di TraCI né WAUT. Effetto collaterale utile: siccome il programma originale resta caricato, in sumo-gui si può passare da uno all'altro col tasto destro sul semaforo, confrontandoli a occhio.

Verifica

Su tutti i 577 semafori delle tre zone: `programID` nuovo, offset e tipo preservati, stati di fase identici all'originale, nessuna durata nulla o negativa, ciclo che passa da 90 s a 120 s. Zero errori.

`sumo_visualize.py` aggiunge da solo il riferimento all'additional-file nel `.sumocfg` se il file esiste; altrimenti la simulazione parte come prima. La webapp eredita tutto perché avvia SUMO dallo stesso script.

Punto 4 — Confronto in SUMO contro la baseline

File: `scripts/compare_sumo.py` → produce `sumo_comparison/{results.json, report.md}`

A cosa serve

L'ottimizzazione del punto 2 *stima* il guadagno con una formula (Webster) che tratta ogni incrocio come **isolato**. Il punto 4 **verifica quella previsione in simulazione**: mette davvero ~45 veicoli in strada dentro SUMO e misura col cronometro quanto ci mettono, prima e dopo. La metrica non viene più dallo stesso modello che ha prodotto l'ottimizzazione, quindi non c'è circolarità fra criterio di ottimizzazione e criterio di giudizio.

Come funziona

Due simulazioni identiche per zona, diverse **solo** nel programma semaforico (baseline = programma originale del `net.xml`; ottimizzato = programma `optimized` dell'additional-file). Per un confronto pulito: stessa domanda O-D del punto 2, **stesse rotte** precalcolate e riusate nei due run, stesso seed e stessi istanti di partenza, teletrasporti disattivati (altrimenti i veicoli bloccati sparirebbero e le attese risulterebbero più basse).

Risultati misurati

Zona	Veicoli	Tempo di viaggio	Attesa ai semafori	Tempo perso
piccola	46	-6,6 %	-68,5 %	-36,5 %
media	45	-8,5 %	-23,1 %	-22,3 %
grande	43	+2,4 %	+10,1 %	+5,6 %

Il risultato più interessante: su "grande" l'ottimizzazione peggiora

Su piccola e media la simulazione conferma il guadagno previsto. Su **grande no**: il piano che il punto 2 stimava migliore del 31 % risulta **peggiore del 10 %** sull'attesa. Non è un bug ed

è riportato come tale: la formula di Webster descrive un incrocio isolato, ipotesi che cade in una rete densa dove le code si propagano fra incroci adiacenti, gli offset non vengono ricalibrati e solo una minoranza di semafori viene ottimizzata. Prova a sostegno: ripetendo il confronto su grande con traffico più leggero il segno si inverte, quindi il degrado emerge **sotto congestione**, cioè dove le ipotesi di Webster valgono meno. Senza il punto 4 il progetto avrebbe riportato come acquisito un guadagno che in simulazione non si verifica.

Replanning — chiusure stradali e ricalcolo del percorso

File: `webapp/app.py` (endpoint `/api/replan`) + `webapp/templates/index.html`

Cosa fa

Nella webapp si possono marcare **strade e incroci come non percorribili** (lavori, incidenti, blocchi) cliccandoli sulla mappa, e chiedere di ricalcolare il percorso. Il ricalcolo non riparte dall'origine: simula un veicolo **già in viaggio** che trova la strada chiusa, quindi ripianifica **dal nodo immediatamente precedente** alla chiusura, cioè l'ultimo punto raggiungibile.

Come funziona

- Si individua il primo elemento bloccato lungo il piano corrente e si ripianifica da lì al goal, escludendo archi e nodi chiusi.
- Il nodo di provenienza viene passato ad ENHSP come `prev` , così la prima svolta dopo la deviazione ha il costo reale (ripartendo da metà percorso il veicolo ha già un orientamento).
- Una strada chiusa lo è in **entrambi i sensi**; chiudere un incrocio equivale a chiudere tutte le strade che vi confluiscono.
- Se le chiusure non toccano il percorso attuale, il sistema lo rileva e non ricalcola nulla.

La parte grafica

Appena si attiva la modalità chiusura, il percorso viene colorato in base a una BFS calcolata **nel browser** (istantanea): in **giallo** i tratti chiudibili (esiste una deviazione), in **rosso scuro** quelli critici (chiuderli isolerebbe la destinazione). Cliccando un tratto critico l'avviso compare subito, senza attendere il server. Dopo il ricalcolo la mappa mostra insieme: il piano originale sbiadito, il tratto **già percorso** in verde, il punto di ricalcolo con un'icona pulsante, e la **deviazione** in ambra con tratteggio animato. Un pannello riepiloga distanza, tempo e semafori prima e dopo.

Un limite intrinseco (importante)

Con pochi nodi il sottografo estratto è quasi un *albero*: molte strade sono l'unico collegamento verso la destinazione, quindi chiuderle la isola e il ricalcolo fallisce legittimamente. Misurato sulla zona media: con 50 nodi 0 tratti su 18 hanno un'alternativa, con 120 nodi 5 su 9, con 300 nodi 14 su 45. La zona **piccola** (centro storico, vie a senso unico) resta quasi ad albero anche con tutti i nodi: non è adatta al replanning. Per provarlo conviene la zona **media** con parecchi nodi.

Contorno tecnico sistemato lungo il percorso

- **Ritardi semaforici realistici** in `build_problems.py` e nella webapp, al posto dei 30 s fissi.
- **Fix del mapping nodo → junction** per SUMO: `netconvert` semplifica la rete diversamente dal grafo contratto, quindi un nodo PDDL può non esistere come junction; ora si cercano anche i membri dei cluster e si ripiega sugli estremi mappabili del piano. Senza questo, la visualizzazione SUMO falliva del tutto sulle zone media e grande.
- **Fix della vista SUMO dopo il replanning**: si salva per SUMO solo il nuovo percorso, così l'auto nasce dove avviene il ricalcolo (prima nasceva fuori inquadratura).
- **Gestione memoria/tempo di ENHSP**: heap configurabile (`ENHSP_HEAP` , default 6 GB), timeout rimosso (`ENHSP_TIMEOUT`), tetto sui nodi configurabile (`MAX_SOLVABLE_NODES` , default 1000). La **visualizzazione** della mappa non ha limiti: la zona media mostra tutti i suoi 939 nodi; il tetto vale solo per la **risoluzione** con ENHSP, perché il grounding delle azioni a 3 argomenti è pesante sui grafi grandi.
- **Riordino del repository**: nuova cartella `docs/` , eliminata `per_professore/` (104 MB, quasi tutto un ambiente virtuale), `.gitignore` aggiornato per i file di configurazione SUMO (che contengono percorsi assoluti e si rigenerano da soli).

In sintesi

Il progetto parte da dati stradali reali di Dublino e costruisce problemi di navigazione in PDDL+. Su questa base sono stati aggiunti quattro passi più una funzione interattiva:

- 1 **Estrazione da SUMO** — si tirano fuori dai file di SUMO i semafori veri e gli angoli di svolta, e si calcolano tempi realistici: attesa media ai semafori (formula di Webster, ciclo 120 s di Dublino) e tempo di svolta (angolo diviso 20 °/s). Questi tempi entrano nel modello PDDL+ al posto dei valori fissi di prima.
- 1 **Ottimizzazione** — si cercano durate di verde migliori; la stima dà un guadagno del 30–37 % sul tempo totale.
- 1 **Iniezione in SUMO** — le durate ottimizzate vengono scritte in un file che SUMO carica come programma semaforico attivo, così si possono vedere in simulazione.
- 1 **Confronto in SUMO** — due simulazioni (semafori originali vs ottimizzati) misurano il guadagno reale. Risultato onesto: migliora su piccola (−68 % di attesa) e media (−23 %), ma **peggiora su grande (+10 %)**, perché la formula usata per ottimizzare ignora le code fra incroci. È il punto 4 a rivelarlo.

Replanning — nella webapp si possono chiudere strade e incroci cliccandoli sulla mappa; il percorso viene ricalcolato dal punto precedente al blocco, simulando un'auto già in viaggio. L'interfaccia segnala in giallo le strade chiudibili e in rosso quelle che isolerebbero la destinazione, e mostra la deviazione con un confronto dei costi.

In una frase: **abbiamo reso i tempi del modello realistici partendo da SUMO, ottimizzato i semafori, verificato in simulazione che l'ottimizzazione non è sempre un guadagno, e aggiunto la possibilità di ripianificare il percorso quando una strada è chiusa.**